

EMS Backend

Exports & Permissions Deep Dive

How CSV / Excel / JSON / PDF and permissions moved to the server

Companion Doc v1.0 · Hostinger 68d32f4

Audience	Backend + UI engineers
----------	------------------------

Live API	ems-api.saqibsaeed.cloud
----------	--------------------------

Date	2026-07-22
------	------------

Companion to	EMS_BACKEND_TECHNICAL_DOCUMENTATION.pdf
--------------	---

Table of Contents

EMS — Backend Exports & Permissions Deep Dive

1. Why this document exists
2. Part A — Moving CSV / Excel / JSON / PDF to the backend
 - 2.1 Before vs after (mental model)
 - 2.2 End-to-end timeline (minute detail)
 - 2.3 Libraries — what / why / how (every one)
 - 2.4 Per-format file contract
 - 2.5 API surface (only these)
 - 2.6 Source map (do not invent new paths)
 - 2.7 What the frontend must delete / change
 - 2.8 Live curl examples (Hostinger)
3. Part B — Moving permissions to the backend (source of truth)
 - 3.1 Before vs after
 - 3.2 Permission catalogue (exact keys)
 - 3.3 Default grants by role (day-1)
 - 3.4 Decision algorithm (`hasPermission`)
 - 3.5 Where permissions attach on login
 - 3.6 Dual gate on exports (role AND permission)
 - 3.7 Settings matrix (admin UI)
 - 3.8 Source map — permissions
 - 3.9 Frontend contract for permissions
4. How the two moves work together
5. Verification checklist
6. Related docs
7. Revision

EMS — Backend Exports & Permissions Deep Dive

Separate companion PDF to the main Technical Documentation.
Focus: how CSV / Excel / JSON / PDF generation moved to the **backend**, every library involved, and how **permissions** became backend source-of-truth.
Deployed: Hostinger `ems-backend` commit **68d32f4** (2026-07-19).
Live API: <https://ems-api.saqibsaeed.cloud/api/v1>

1. Why this document exists

Two UI-side problems were fixed on the backend in the July 2026 Hostinger hardening pass:

Problem	Old (frontend)	New (backend)
Operational exports	Browser built CSV / Excel / PDF with <code>Blob</code> , client libraries, or string concat	Server queues a job, writes the file from live DB rows, stores it, returns download
Permission gates	FE hard-coded role→permission matrices; empty JWT <code>permissions[]</code> broke gates	<code>auth.policy.js</code> is SoT; JWT carries resolved permissions; <code>requirePermission</code> enforces on every mutating/export route

This PDF explains **every library**, **every step**, and **every file** involved — not a summary.

2. Part A — Moving CSV / Excel / JSON / PDF to the backend

2.1 Before vs after (mental model)

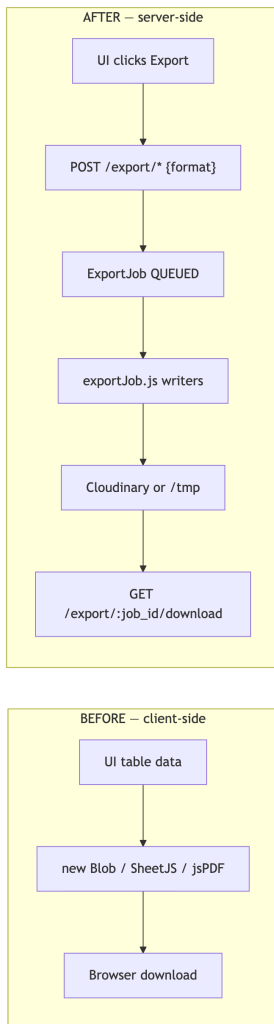


Diagram 1

Why the old way was wrong

1. **Incomplete / stale data** — UI only exported what was already loaded in the grid (pagination truncated rows).
2. **No tenant enforcement on file content** — anything in memory could be dumped.
3. **No permission enforcement on the bytes** — a crafty client could build a file without calling a gated API.
4. **Inconsistent columns** — each screen invented its own CSV headers.
5. **Container-hostile** — large client PDFs burned browser memory; no durable audit of “who exported what”.

Why the new way is correct

1. Prisma reads are **tenant-scoped** in `export.repository.js`.
2. Auth is **role + permission** on the route before any job is created.
3. One writer path (`generateExportFile`) for all four formats.
4. Durable storage via Cloudinary (`raw + authenticated`) so Docker restarts do not lose files.
5. Audit-friendly: every export creates an `ExportJob` row with `job_id`, type, format, status.

2.2 End-to-end timeline (minute detail)

Step	Who	What happens	Code
0	FE	User clicks Export → chooses format	Must not use <code>Blob</code>
1	FE → BE	<code>POST /api/v1/export/employees</code> body { "format": "pdf" }	<code>export.routes.js</code>
2	Middleware	JWT verify → <code>authorize(['HR_ADMIN'])</code> → <code>requirePermission('employees:export')</code>	<code>authenticate.js</code> + <code>auth.policy.js</code>
3	Controller	Zod-parse body (<code>csv\ excel\ json\ pdf</code>)	<code>export.controller.js</code> + <code>export.validator.js</code>
4	Service	<code>uuidv4()</code> → <code>createExportJob</code> status <code>QUEUED</code>	<code>export.service.js</code>
5	Service	Return 202 { <code>job_id</code> , <code>status: "QUEUED"</code> , <code>estimated_completion_time</code> } immediately	same
6	Service	<code>setImmediate(() => exportEmployees(...))</code> — non-blocking worker	same
7	Worker	<code>getEmployeesForExport(tenantId, filters)</code> — live Postgres	<code>export.repository.js</code>
8	Worker	<code>generateExportFile(rows, 'employees', format, jobId)</code>	<code>exportJob.js</code>
9a	CSV path	<code>flattenObject</code> → <code>escapeCSV</code> → <code>writeFileSync</code>	<code>generateCSV</code>
9b	Excel path	ExcelJS workbook, frozen header <code>#4F46E5</code> , alt rows <code>#F0F0FF</code>	<code>generateExcel</code>
9c	JSON path	<code>JSON.stringify(data, null, 2)</code>	<code>generateJSON</code>
9d	PDF path	PDFKit landscape A4, indigo header bar, max 8 columns	<code>generatePDF</code>
10	Persist	Read file buffer → Cloudinary upload <code>resourceType: 'raw', type: 'authenticated'</code> → store <code>cloudinary://{publicId}</code>	<code>persistSuccess</code>
11	Fallback	If Cloudinary fails → keep local <code>/tmp/exports/{jobId}.{ext}</code> URL	same
12	Status	<code>ExportJob</code> → <code>SUCCESS</code> + <code>fileUrl</code>	<code>repository</code>
13	FE	<code>GET /export/{job_id}/download</code> with Bearer	<code>controller</code>
14a	Download	If <code>cloudinary://{}</code> → 302 to signed URL (300s)	<code>getSignedDocumentUrl</code>
14b	Download	Else stream file from disk with <code>Content- Disposition: attachment</code>	<code>createReadStream</code>
15	Not ready	If status ≠ <code>SUCCESS</code> → 200 JSON status (poll again)	<code>controller</code>

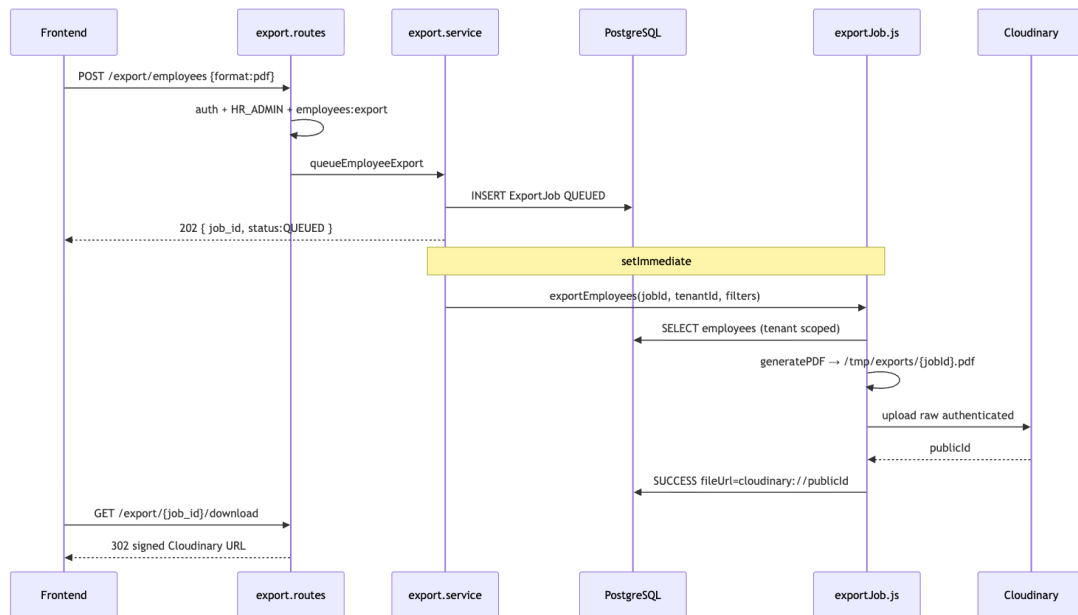


Diagram 2

2.3 Libraries — what / why / how (every one)

Library	npm	What it is	Why we use it	How EMS uses it
ExcelJS	exceljs@^4.4.0	Pure-Node XLSX writer/reader	Styled workbooks without spawning Microsoft Excel or heavy browser SheetJS	generateExcel() — frozen header, indigo fill FF4F46E5 , white bold font, alternating FFF0F0FF , auto column width capped at 40, footer "Total: N records"
PDFKit	pdfkit@0.19.1	Streaming PDF generator	Server PDF without Puppeteer/Chrome; tiny footprint for table exports	generatePDF() — layout: 'landscape' , size: 'A4' , header bar #4F46E5 , first 8 columns only (note if more), paginate when doc.y near bottom
Cloudinary SDK	cloudinary@2.10.0	Object storage + signed delivery	Durable files across Docker restarts; private authenticated assets	uploadToCloudinary(buffer, { resourceType: 'raw', type: 'authenticated', folder: ems/{tenantId}/exports, publicId: jobId }) ; download via private_download_url (~300s)
Node fs	built-in	Filesystem	Stage bytes before upload; fallback when Cloudinary down	writeFileSync / readFileSync under config.exportsDir or /tmp/exports
Prisma	@prisma/client	ORM	Tenant-safe queries + ExportJob lifecycle	getEmployeesForExport / attendance / leave + create/update job status
uuid	uuid@14	UUID v4	Opaque job_id clients poll	uuidv4() in queue*Export

Library	npm	What it is	Why we use it	How EMS uses it
Zod	zod@^3.23	Schema validation	Reject bad format / missing date ranges before queue	export.validator.js enums csv\ excel\ json\ pdf
Fastify	fastify@4	HTTP framework	Route hooks for auth + permission	onRequest: [authenticate, authorize, requirePermission]
Pino	pino	Structured logs	Trace queue / success / Cloudinary failure	type: 'export_queued' \ 'export_completed' \ 'export_uploaded_cloudinary'

ExcelJS — deeper

- Creates `Workbook` → worksheet titled from export type (`Employees` , `Attendance` , `Leave`).
- `views: [{ state: 'frozen', ySplit: 1 }]` keeps header visible while scrolling in Excel.
- Nested objects are flattened (`department.name` → `department.name`) so one flat table is written.
- Empty dataset → single cell `No data available` (still a valid XLSX).

PDFKit — deeper

- Collects PDF chunks via `doc.on('data')` , concatenates, writes once on `end` .
- Landscape chosen so more columns fit; still capped at 8 for readability.
- Doc explicitly tells operators: *use Excel/CSV for full width* when columns > 8.
- Alternating row tint `#F0F0FF` matches Excel branding.

Cloudinary — deeper

- Prefix stored in DB: `cloudinary://` + `public_id` (not a permanent public HTTP URL).
- Delivery type `authenticated` means anonymous URL fetch returns 401.
- Download route mints a **short-lived signed** URL; FE should follow redirects (`curl -L` / browser).
- If signing fails → fall back to local disk stream if file still on container.

2.4 Per-format file contract

format body	On-disk / download name	MIME	Generator	Notes
csv	{jobId}.csv	text/csv	generateCSV	RFC quoting for <code>,</code> <code>"</code> newlines; empty file if 0 rows
excel	{jobId}.xlsx	OOXML spreadsheet	generateExcel	ExcelJS; not CSV-with-.xls
json	{jobId}.json	application/json	generateJSON	Pretty nested objects (not flattened)
pdf	{jobId}.pdf	application/pdf	generatePDF	Landscape A4 PDFKit

2.5 API surface (only these)

Method	Path	Auth	Body / query	HTTP
POST	/api/v1/export/employees	HR_ADMIN + employees:export	format , optional department_id , status , include_archived	202
POST	/api/v1/export/attendance	same	requires from_date , to_date + format	202
POST	/api/v1/export/leave	same	requires from_date , to_date + format	202
GET	/api/v1/export/:job_id/download	same	—	200 stream / 302 signed / 200 status JSON
GET	/api/v1/export/list	authenticated	page , limit , status	200

202 body:

```
{
  "success": true,
  "data": {
    "job_id": "01a08046-e986-4a97-bbfe-68c3f042e8a3",
    "status": "QUEUED",
    "estimated_completion_time": 2
  }
}
```

2.6 Source map (do not invent new paths)

Layer	File
Routes + dual auth	src/modules/export/export.routes.js
HTTP + MIME + 302	src/modules/export/export.controller.js
Queue + setImmediate	src/modules/export/export.service.js
Prisma + ExportJob	src/modules/export/export.repository.js
Zod	src/modules/export/export.validator.js
CSV/Excel/JSON/PDF + Cloudinary persist	src/jobs/exportJob.js
Cloudinary helpers	src/utils/cloudinary.js
Permission gate	src/modules/auth/auth.policy.js

2.7 What the frontend must delete / change

1. Delete `new Blob([csvString])` , client SheetJS, client jsPDF for **employees / attendance / leave** operational exports.

2. Call `POST /export/*` → store `job_id` → open/poll `GET /export/:job_id/download`.
3. Never use legacy `/files/:jobId`.
4. Hide Export buttons unless `/auth/me` includes `employees:export` (and role allows).
5. Contract file: `docs/UI_CONTRACT_server_exports_permissions_realtime_logs.md`.

2.8 Live curl examples (Hostinger)

```

BASE=https://ems-api.saqibsaeed.cloud/api/v1
TOKEN=$(curl -s -X POST $BASE/auth/login \
  -H 'content-type: application/json' -H 'x-tenant-key: acme-corp-001' \
  -d '{"email":"hr@acme.test","password":"Password123!"}' \
  | python3 -c 'import sys,json; print(json.load(sys.stdin)["data"]["accessToken"])')

# CSV
curl -s -X POST $BASE/export/employees -H "Authorization: Bearer $TOKEN" \
  -H 'content-type: application/json' -d '{"format":"csv"}'

# Excel
curl -s -X POST $BASE/export/employees -H "Authorization: Bearer $TOKEN" \
  -H 'content-type: application/json' -d '{"format":"excel","status":"ACTIVE"}'

# PDF
curl -s -X POST $BASE/export/employees -H "Authorization: Bearer $TOKEN" \
  -H 'content-type: application/json' -d '{"format":"pdf"}'

# JSON attendance (dates required)
curl -s -X POST $BASE/export/attendance -H "Authorization: Bearer $TOKEN" \
  -H 'content-type: application/json' \
  -d '{"format":"json","from_date":"2026-07-01","to_date":"2026-07-19"}'

# Download (follow 302)
curl -sL -o /tmp/ems-export.bin -H "Authorization: Bearer $TOKEN" \
  "$BASE/export/<JOB_ID>/download"

# Employee denied
EMP=$(curl -s -X POST $BASE/auth/login -H 'content-type: application/json' \
  -H 'x-tenant-key: acme-corp-001' \
  -d '{"email":"priya@acme.test","password":"Password123!"}' \
  | python3 -c 'import sys,json; print(json.load(sys.stdin)["data"]["accessToken"])')
curl -s -o /dev/null -w '%{http_code}\n' -X POST $BASE/export/employees \
  -H "Authorization: Bearer $EMP" -H 'content-type: application/json' -d '{"format":"csv"}'
# → 403

```

Proven live (2026-07-19 via Vercel BFF): HR → 202 QUEUED; Employee → 403 FORBIDDEN.

3. Part B — Moving permissions to the backend (source of truth)

3.1 Before vs after

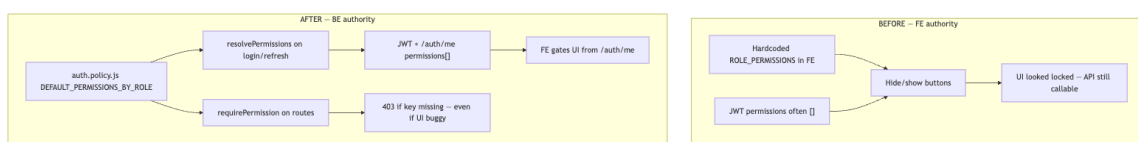


Diagram 3

Why FE-as-authority failed

1. Empty `permissions[]` in JWT made FE hide everything or fall back to wrong constants.
2. Changing the matrix in Settings UI did not change what the API allowed until BE enforced keys.
3. Security cannot depend on CSS/ `if (role === 'HR')` in React — anyone can call `curl`.

What “moved to backend” means

1. **Canonical defaults** live in `src/modules/auth/auth.policy.js` → `DEFAULT_PERMISSIONS_BY_ROLE`.
2. **Login / refresh / me** call `resolvePermissions(user)` :
 - Prefer explicit `RolePermission` rows from DB.
 - Else fall back to the default matrix for `memberType`.
3. **Route enforcement** via `requirePermission('employees:export')` (and write/delete keys on employee mutations).
4. **Settings API** still lets SUPER_ADMIN PATCH the matrix; seed runs **once** per tenant so customizations are never overwritten.
5. FE must **display** from `/auth/me` and **never** treat local constants as authority.

3.2 Permission catalogue (exact keys)

Key	Module	Meaning
<code>employees:read</code>	employees	View employees
<code>employees:write</code>	employees	Create / update
<code>employees:delete</code>	employees	Soft delete
<code>employees:export</code>	employees	Queue/download exports
<code>departments:read</code>	departments	View
<code>departments:write</code>	departments	Manage
<code>attendance:read</code>	attendance	View
<code>attendance:write</code>	attendance	Mutate
<code>leave:read</code>	leave	View
<code>leave:request</code>	leave	Request
<code>leave:approve</code>	leave	Approve / deny
<code>analytics:read</code>	analytics	View analytics
<code>audit:read</code>	audit	View audit logs
<code>permissions:manage</code>	settings	Edit role matrix

3.3 Default grants by role (day-1)

Role	Defaults
SUPER_ADMIN	All keys including <code>permissions:manage</code> . Also bypasses <code>hasPermission</code> checks.
HR_ADMIN	employees CRUD+export, departments r/w, attendance r/w, leave r/request/approve, analytics, audit — not <code>permissions:manage</code>
MANAGER	employees:read, departments:read, attendance r/w, leave r/request/approve, analytics:read — no export

Role	Defaults
EMPLOYEE	employees:read, departments:read, attendance r/w, leave read/request — no export
AUDITOR	read-only employees/departments/attendance/leave + analytics + audit

3.4 Decision algorithm (`hasPermission`)

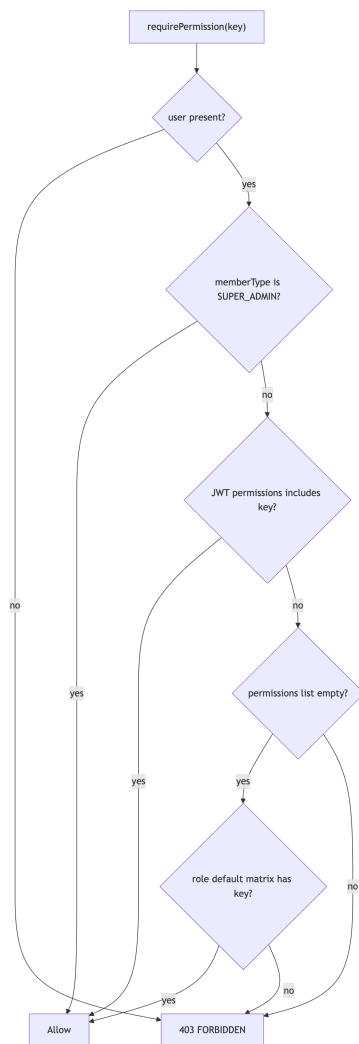


Diagram 4

This empty-token fallback exists so **old JWTs minted before seed** do not lock out HR on day-1. After refresh, tokens carry the full list.

3.5 Where permissions attach on login

1. User authenticates → load user + roles/permissions relations.
2. `resolvePermissions(user)` → string array.
3. Array embedded in access token claims **and** returned from `GET /auth/me`.
4. Every protected export/write route re-checks with `requirePermission` using `request.user` from JWT.

3.6 Dual gate on exports (role AND permission)

```
// export.routes.js – both required
onRequest: [
  authenticate,
  authorize(['HR_ADMIN']), // role enum
  requirePermission('employees:export'), // fine-grained key
]
```

Actor	Role OK?	<code>employees:export</code> ?	Result
HR_ADMIN with default grants	yes	yes	202
HR_ADMIN after admin removes export key + refreshed token	yes	no	403
EMPLOYEE	no	no	403 (authorize or permission)
SUPER_ADMIN	bypass	bypass	allowed

3.7 Settings matrix (admin UI)

Method	Path	Who	Effect
GET	<code>/settings/roles-permissions</code>	SUPER_ADMIN	Read matrix
PATCH	<code>/settings/roles-permissions</code>	SUPER_ADMIN + <code>permissions:manage</code>	Update RolePermission rows

After PATCH: FE must **refresh token or re-login**. Until then UI may look stale; **backend still enforces the new grants** on the next request that carries an updated JWT. Requests with old JWT keep old grants until expiry/refresh.

Seed helper: `ensureTenantRolePermissionDefaults` — runs once (`setting` flag `role_permissions_defaults_seeded`); never wipes admin customizations later.

3.8 Source map — permissions

Concern	File
Defaults + <code>requirePermission</code> + <code>hasPermission</code>	<code>src/modules/auth/auth.policy.js</code>
<code>resolvePermissions</code> + catalogue seed	<code>src/modules/auth/auth.service.js</code>
Role authorize middleware	<code>src/middleware/authenticate.js</code>
Export dual gate	<code>src/modules/export/export.routes.js</code>
Employee write/delete/export gates	<code>src/modules/employees/employees.routes.js</code>
Settings matrix API	<code>src/modules/settings/settings.*.js</code>

3.9 Frontend contract for permissions

1. **Do not** hardcode default matrices as authority.
2. Gate buttons/menus from `/auth/me` → `permissions[]` (+ `memberType`).
3. After PATCH roles-permissions → force refresh/login.
4. Expect **403** with body `{ requiredPermission, userRole }` when key missing — show a clear message, do not retry as Blob export.

4. How the two moves work together

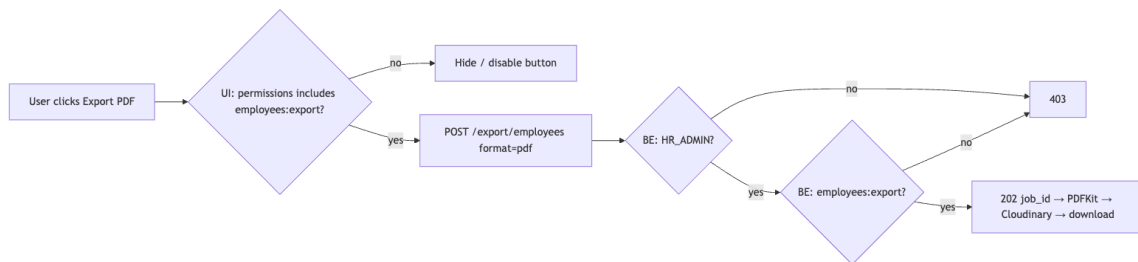


Diagram 5

- UI gate = UX.
- Backend gate = security.
- File bytes = always generated on server with tenant-scoped Prisma + ExcelJS/PDFKit.

5. Verification checklist

#	Check	Expected
1	HR POST <code>.../export/employees {format:csv excel json pdf}</code>	202 + <code>job_id</code>
2	GET <code>.../export/{job_id}/download</code> after ~1-2s	file or 302
3	EMP same POST	403
4	Remove <code>employees:export</code> from HR via settings → refresh token → export	403
5	Restore permission → export	202 again
6	<code>/auth/me</code> for HR	<code>permissions</code> includes <code>employees:export</code>
7	No client <code>Blob</code> for operational exports in FE	code review

6. Related docs

Doc	Use
<code>docs/EMS_BACKEND_TECHNICAL_DOCUMENTATION.pdf</code>	Full system tech doc (v3.1) — §10.4 / §10.5
<code>docs/UI_CONTRACT_server_exports_permissions_realtime_logs.md</code>	FE must-do contract

Doc	Use
docs/BACKEND_CHANGELOG_hostinger_hardening_2026-07-19.md	Changelog
docs/LIVE_UI_ROLE_MATRIX_VERCEL_2026-07-19.md	Live HR 202 / EMP 403 proof
docs/UI_TEAM_HANDOFF_HOSTINGER_HARDENING_2026-07-19.md	Handoff index

7. Revision

Date	Version	Notes
2026-07-22	1.0	Dedicated deep-dive PDF: backend exports (all libraries + steps) + permissions SoT migration